

Terraform Labs

From Installation to Multi-Cloud Automation — AWS, Azure & Multi-Cloud

Prepared for classroom / self-paced lab use

Covers: Terraform installation → AWS labs → Azure labs → Multi-cloud labs → Validation

Contents

- 0. Prerequisites & Terraform Installation
- AWS Lab 1 (Basic): Your First EC2 Instance with Terraform
- AWS Lab 2 (Intermediate): VPC, Security Group & EC2 with Variables
- AWS Lab 3 (Advanced): Highly Available Web App (ALB + ASG + RDS + Remote State)
- Azure Lab 1 (Basic): Your First Azure VM with Terraform
- Azure Lab 2 (Intermediate): VNet, NSG & VM with Public IP
- Azure Lab 3 (Advanced): VM Scale Set + Load Balancer + Azure SQL + Remote State
- Multi-Cloud Lab 1 (Basic): Single Config, Two Clouds
- Multi-Cloud Lab 2 (Advanced): Active-Passive DR Across AWS & Azure
- Final Validation Checklist

0. Prerequisites & Terraform Installation

This section gets a Linux box (an AWS EC2 instance or an Azure VM — either works identically for these labs) ready to run Terraform, the AWS CLI, and the Azure CLI.

0.1 Create the Lab Machine

Option A — AWS EC2:

- Launch an EC2 instance: Amazon Linux 2023 or Ubuntu 22.04, t2.micro/t3.micro
- Open port 22 (SSH) in the security group
- Attach an IAM role with programmatic access, or plan to configure access keys

Option B — Azure VM:

- Create a VM: Ubuntu 22.04 LTS, Standard_B1s
- Open port 22 (SSH) in the Network Security Group

Connect to whichever machine you chose:

```
ssh -i your-key.pem ec2-user@<public-ip>      # AWS example
ssh azureuser@<public-ip>                     # Azure example
```

0.2 Install Terraform

1. Update packages and install prerequisites:

```
sudo yum update -y                # Amazon Linux
sudo yum install -y yum-utils      # Amazon Linux
# --- OR, on Ubuntu/Debian ---
sudo apt update && sudo apt install -y gnupg software-properties-common curl
```

2. Add HashiCorp's repository and install Terraform:

```
# Amazon Linux / RHEL
sudo yum-config-manager --add-repo
https://rpm.releases.hashicorp.com/AmazonLinux/hashicorp.repo
sudo yum install -y terraform
# --- Ubuntu / Debian ---
wget -O- https://apt.releases.hashicorp.com/gpg | sudo gpg --dearmor -o
/usr/share/keyrings/hashicorp-archive-keyring.gpg
echo "deb [signed-by=/usr/share/keyrings/hashicorp-archive-keyring.gpg]
https://apt.releases.hashicorp.com $(lsb_release -cs) main" | sudo tee
/etc/apt/sources.list.d/hashicorp.list
sudo apt update && sudo apt install -y terraform
```

3. Verify installation:

```
terraform -version
```

✓ Validation Steps

4. Command returns a Terraform version (e.g. Terraform v1.9.x) with no errors.

0.3 Install & Configure Cloud CLIs

AWS CLI

```
curl "https://awscli.amazonaws.com/awscli-exe-linux-x86_64.zip" -o "awscliv2.zip"
sudo yum install -y unzip && unzip awscliv2.zip && sudo ./aws/install
aws configure      # enter Access Key, Secret Key, region (e.g. ap-south-1), output format
json
```

Azure CLI

```
curl -sL https://aka.ms/InstallAzureCLIDeb | sudo bash # Ubuntu
az login # opens a device-login URL, or use --identity on an Azure VM with a managed
identity
```

✓ Validation Steps

5. ``aws sts get-caller-identity`` returns your AWS account ID and user/role ARN.
6. ``az account show`` returns your Azure subscription details.

LAB 1

AWS — Your First EC2 Instance with Terraform

Difficulty: Basic

Objective: Provision a single EC2 instance using Terraform and understand the core workflow: init → plan → apply → destroy.

Steps

7. Create a project folder and main configuration file:

```
mkdir ~/terraform-lab1 && cd ~/terraform-lab1
touch main.tf
```

8. Add the following to main.tf:

```
terraform {
  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 5.0"
    }
  }
}

provider "aws" {
  region = "ap-south-1"
}

resource "aws_instance" "lab_ec2" {
  ami           = "ami-0f5ee92e2d63afc18"    # Amazon Linux 2023 (ap-south-1)
  instance_type = "t2.micro"

  tags = {
    Name = "Terraform-Lab1-Instance"
  }
}
```

9. Initialize, plan, and apply:

```
terraform init
terraform plan
terraform apply -auto-approve
```

10. Note the instance ID in the apply output.

✓ Validation Steps

11. Run `terraform show` — the aws_instance resource with your tag should appear.
12. In the AWS Console → EC2 → Instances, confirm “Terraform-Lab1-Instance” is running.
13. Run `aws ec2 describe-instances --filters "Name=tag:Name,Values=Terraform-Lab1-Instance"` and confirm the state is `running`.
14. Run `terraform destroy -auto-approve` and confirm the instance disappears from the console.

LAB 2

AWS — VPC, Security Group & EC2 with Variables

Difficulty: Intermediate

Objective: Build a small custom network (VPC + subnet + security group) and launch an EC2 instance into it, using variables and outputs instead of hardcoded values.

Steps

15. Create the project and three files: variables.tf, main.tf, outputs.tf:

```
mkdir ~/terraform-lab2 && cd ~/terraform-lab2
touch variables.tf main.tf outputs.tf
```

16. variables.tf:

```
variable "region" {
  default = "ap-south-1"
}

variable "vpc_cidr" {
  default = "10.0.0.0/16"
}

variable "subnet_cidr" {
  default = "10.0.1.0/24"
}

variable "instance_type" {
  default = "t2.micro"
}
```

17. main.tf:

```
provider "aws" {
  region = var.region
}

resource "aws_vpc" "lab_vpc" {
  cidr_block = var.vpc_cidr
  tags      = { Name = "Lab2-VPC" }
}

resource "aws_subnet" "lab_subnet" {
  vpc_id            = aws_vpc.lab_vpc.id
  cidr_block        = var.subnet_cidr
  map_public_ip_on_launch = true
  availability_zone  = "ap-south-1a"
  tags = { Name = "Lab2-Subnet" }
}

resource "aws_internet_gateway" "lab_igw" {
  vpc_id = aws_vpc.lab_vpc.id
}

resource "aws_route_table" "lab_rt" {
  vpc_id = aws_vpc.lab_vpc.id
  route {
    cidr_block = "0.0.0.0/0"
    gateway_id = aws_internet_gateway.lab_igw.id
  }
}

resource "aws_route_table_association" "lab_rta" {
```

```

    subnet_id      = aws_subnet.lab_subnet.id
    route_table_id = aws_route_table.lab_rt.id
}

resource "aws_security_group" "lab_sg" {
  name     = "lab2-sg"
  vpc_id   = aws_vpc.lab_vpc.id

  ingress {
    from_port = 22
    to_port   = 22
    protocol  = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }
  egress {
    from_port = 0
    to_port   = 0
    protocol  = "-1"
    cidr_blocks = ["0.0.0.0/0"]
  }
}

resource "aws_instance" "lab_ec2" {
  ami                = "ami-0f5ee92e2d63afc18"
  instance_type      = var.instance_type
  subnet_id          = aws_subnet.lab_subnet.id
  vpc_security_group_ids = [aws_security_group.lab_sg.id]
  tags               = { Name = "Lab2-EC2" }
}

```

18.outputs.tf:

```

output "instance_public_ip" {
  value = aws_instance.lab_ec2.public_ip
}

output "vpc_id" {
  value = aws_vpc.lab_vpc.id
}

```

19.Apply:

```

terraform init
terraform plan
terraform apply -auto-approve

```

Validation Steps

20. Terraform output prints `instance\_public\_ip` and `vpc\_id`.
21. SSH into the instance using the printed public IP to confirm connectivity through your custom VPC.
22. In AWS Console → VPC, confirm the VPC, subnet, route table, and IGW all show “Lab2-” naming.
23. Run `terraform state list` and confirm all 7 resources are tracked.
24. `terraform destroy -auto-approve` to clean up and confirm no orphaned resources remain.

LAB 3

AWS — Highly Available Web App (ALB + Auto Scaling + RDS + Remote State)

Difficulty: Advanced

Objective: Combine everything into a production-style pattern: remote state in S3 with a DynamoDB lock, a Launch Template, an Auto Scaling Group behind an Application Load Balancer, and an RDS MySQL backend.

Steps

25. Create the remote state backend first (one-time, separate mini-project):

```
resource "aws_s3_bucket" "tf_state" {
  bucket = "my-terraform-state-<unique-suffix>"
}

resource "aws_dynamodb_table" "tf_lock" {
  name           = "terraform-lock"
  billing_mode   = "PAY_PER_REQUEST"
  hash_key       = "LockID"
  attribute {
    name = "LockID"
    type = "S"
  }
}
```

26. In the main lab project, point Terraform at that backend:

```
terraform {
  backend "s3" {
    bucket = "my-terraform-state-<unique-suffix>"
    key    = "lab3/terraform.tfstate"
    region = "ap-south-1"
    dynamodb_table = "terraform-lock"
  }
}
```

27. Define a Launch Template and Auto Scaling Group (building on the earlier AWS Console walkthrough, now as code):

```
resource "aws_launch_template" "web_lt" {
  name_prefix = "lab3-web-"
  image_id    = "ami-0f5ee92e2d63afc18"
  instance_type = "t2.micro"
  user_data = base64encode(<<-EOF
    #!/bin/bash
    echo "hello from $(hostname)" > /var/www/html/index.html
  EOF
  )
}

resource "aws_autoscaling_group" "web_asg" {
  desired_capacity = 2
  min_size         = 2
  max_size         = 4
  vpc_zone_identifier = [aws_subnet.public_a.id, aws_subnet.public_b.id]
  target_group_arns   = [aws_lb_target_group.web_tg.arn]

  launch_template {
    id      = aws_launch_template.web_lt.id
    version = "$Latest"
  }
}
```

28. Define an Application Load Balancer and target group:

```
resource "aws_lb" "web_alb" {
  name            = "lab3-alb"
  internal        = false
  load_balancer_type = "application"
  subnets        = [aws_subnet.public_a.id, aws_subnet.public_b.id]
  security_groups = [aws_security_group.alb_sg.id]
}

resource "aws_lb_target_group" "web_tg" {
  name      = "lab3-tg"
  port      = 80
  protocol  = "HTTP"
  vpc_id    = aws_vpc.lab_vpc.id
  health_check { path = "/" }
}

resource "aws_lb_listener" "web_listener" {
  load_balancer_arn = aws_lb.web_alb.arn
  port              = 80
  protocol          = "HTTP"
  default_action {
    type = "forward"
    target_group_arn = aws_lb_target_group.web_tg.arn
  }
}
```

29. Define an RDS MySQL instance in a private subnet:

```
resource "aws_db_instance" "web_db" {
  identifier      = "lab3-db"
  engine          = "mysql"
  instance_class  = "db.t3.micro"
  allocated_storage = 20
  username        = "admin"
  password        = var.db_password
  db_subnet_group_name = aws_db_subnet_group.private.name
  skip_final_snapshot = true
}
```

30. Apply the full stack:

```
terraform init
terraform plan -out=lab3.plan
terraform apply lab3.plan
```

✓ Validation Steps

31. `terraform output alb\_dns\_name` returns a DNS name; `curl <alb\_dns\_name>` returns the user-data web page.
32. Refresh curl a few times / check the target group — requests should be served by both ASG instances (round robin).
33. In EC2 → Auto Scaling Groups, confirm desired=2 and both instances show “healthy” in the target group.
34. Terminate one ASG instance manually and confirm the ASG launches a replacement (same behavior validated earlier from the AWS Console, now fully automated as code).
35. Connect to the RDS endpoint from an EC2 instance in the same VPC to confirm the database is reachable only internally, not from the public internet.
36. Check S3 → your state bucket → confirm `lab3/terraform.tfstate` exists, and DynamoDB → confirm a lock row appears only while apply/destroy is running.
37. `terraform destroy` to tear down the entire stack; confirm ALB, ASG, RDS, and networking resources are all removed.

Azure — Your First Azure VM with Terraform

Difficulty: Basic

Objective: Provision a resource group and a single Linux VM in Azure using Terraform.

Steps

38. Create the project:

```
mkdir ~/terraform-az-lab1 && cd ~/terraform-az-lab1
touch main.tf
```

39. main.tf:

```
terraform {
  required_providers {
    azurerm = {
      source = "hashicorp/azurerm"
      version = "~> 3.0"
    }
  }
}

provider "azurerm" {
  features {}
}

resource "azurerm_resource_group" "lab_rg" {
  name     = "tf-lab1-rg"
  location = "Central India"
}

resource "azurerm_virtual_network" "lab_vnet" {
  name            = "lab1-vnet"
  address_space   = ["10.10.0.0/16"]
  location        = azurerm_resource_group.lab_rg.location
  resource_group_name = azurerm_resource_group.lab_rg.name
}

resource "azurerm_subnet" "lab_subnet" {
  name                 = "lab1-subnet"
  resource_group_name = azurerm_resource_group.lab_rg.name
  virtual_network_name = azurerm_virtual_network.lab_vnet.name
  address_prefixes     = ["10.10.1.0/24"]
}

resource "azurerm_public_ip" "lab_pip" {
  name             = "lab1-pip"
  resource_group_name = azurerm_resource_group.lab_rg.name
  location         = azurerm_resource_group.lab_rg.location
  allocation_method = "Static"
}

resource "azurerm_network_interface" "lab_nic" {
  name            = "lab1-nic"
  location        = azurerm_resource_group.lab_rg.location
  resource_group_name = azurerm_resource_group.lab_rg.name

  ip_configuration {
    name                       = "internal"
    subnet_id                 = azurerm_subnet.lab_subnet.id
    private_ip_address_allocation = "Dynamic"
    public_ip_address_id       = azurerm_public_ip.lab_pip.id
  }
}
```

```

    }
}

resource "azurerm_linux_virtual_machine" "lab_vm" {
  name                = "lab1-vm"
  resource_group_name = azurerm_resource_group.lab_rg.name
  location            = azurerm_resource_group.lab_rg.location
  size                = "Standard_B1s"
  admin_username      = "azureuser"
  network_interface_ids = [azurerm_network_interface.lab_nic.id]

  admin_ssh_key {
    username   = "azureuser"
    public_key = file("~/ssh/id_rsa.pub")
  }

  os_disk {
    caching              = "ReadWrite"
    storage_account_type = "Standard_LRS"
  }

  source_image_reference {
    publisher = "Canonical"
    offer     = "0001-com-ubuntu-server-jammy"
    sku       = "22_04-lts"
    version   = "latest"
  }
}

```

40. Apply:

```

terraform init
terraform plan
terraform apply -auto-approve

```

✓ Validation Steps

41. `az vm list -g tf-lab1-rg -o table` shows `lab1-vm` as running.
42. SSH into the VM using the public IP shown in the Azure Portal or via `az vm show`.
43. `terraform destroy -auto-approve` removes the resource group and all nested resources.

LAB 2

Azure — VNet, NSG & VM with Public IP

Difficulty: Intermediate

Objective: Add a Network Security Group with explicit inbound rules, and structure the project with variables, so students see how Azure networking concepts map to the AWS VPC/SG concepts from AWS Lab 2.

Steps

44. Extend Lab 1's project (or start fresh) and add variables.tf:

```
variable "location" {
  default = "Central India"
}
variable "vm_size" {
  default = "Standard_B1s"
}
```

45. Add a Network Security Group and associate it with the subnet:

```
resource "azurerm_network_security_group" "lab_nsg" {
  name                = "lab2-nsg"
  location            = var.location
  resource_group_name = azurerm_resource_group.lab_rg.name

  security_rule {
    name                = "Allow-SSH"
    priority            = 100
    direction           = "Inbound"
    access              = "Allow"
    protocol            = "Tcp"
    source_port_range   = "*"
    destination_port_range = "22"
    source_address_prefix = "*"
    destination_address_prefix = "*"
  }
  security_rule {
    name                = "Allow-HTTP"
    priority            = 110
    direction           = "Inbound"
    access              = "Allow"
    protocol            = "Tcp"
    source_port_range   = "*"
    destination_port_range = "80"
    source_address_prefix = "*"
    destination_address_prefix = "*"
  }
}

resource "azurerm_subnet_network_security_group_association" "lab_nsg_assoc" {
  subnet_id            = azurerm_subnet.lab_subnet.id
  network_security_group_id = azurerm_network_security_group.lab_nsg.id
}
```

46. Add a custom_data script so the VM installs and starts nginx on boot:

```
resource "azurerm_linux_virtual_machine" "lab_vm" {
  # ... same fields as Lab 1 ...
  custom_data = base64encode(<<-EOF
    #!/bin/bash
    apt update && apt install -y nginx
    systemctl enable nginx --now
  EOF
)
```

```
)  
}
```

47. Apply:

```
terraform init  
terraform apply -auto-approve
```

✓ Validation Steps

48. `curl http://<public-ip>` returns the default nginx welcome page.

49. In Azure Portal → Networking on the VM, confirm the NSG shows Allow-SSH and Allow-HTTP inbound rules.

50. Attempt to reach any other port (e.g. 8080) and confirm it is blocked by default, illustrating NSG allow-list behavior.

51. `terraform destroy -auto-approve` cleans up all resources.

LAB 3

Azure — VM Scale Set + Load Balancer + Azure SQL + Remote State

Difficulty: Advanced

Objective: Mirror AWS Lab 3's architecture on Azure: a VM Scale Set behind a Load Balancer, an Azure SQL database, and remote state stored in an Azure Storage Account.

Steps

52. Create the remote state backend (Storage Account + container), one-time setup:

```
az group create -n tfstate-rg -l centralindia
az storage account create -n tfstatestore<unique> -g tfstate-rg -l centralindia --sku
Standard_LRS
az storage container create -n tfstate --account-name tfstatestore<unique>
```

53. Point the lab project at that backend:

```
terraform {
  backend "azurerm" {
    resource_group_name = "tfstate-rg"
    storage_account_name = "tfstatestore<unique>"
    container_name      = "tfstate"
    key                 = "lab3.terraform.tfstate"
  }
}
```

54. Define a Load Balancer with a public frontend IP:

```
resource "azurerm_lb" "web_lb" {
  name                = "lab3-lb"
  location            = var.location
  resource_group_name = azurerm_resource_group.lab_rg.name
  sku                 = "Standard"

  frontend_ip_configuration {
    name                = "PublicIPAddress"
    public_ip_address_id = azurerm_public_ip.lb_pip.id
  }
}

resource "azurerm_lb_backend_address_pool" "web_pool" {
  loadbalancer_id = azurerm_lb.web_lb.id
  name            = "BackendPool"
}

resource "azurerm_lb_probe" "web_probe" {
  loadbalancer_id = azurerm_lb.web_lb.id
  name            = "http-probe"
  port            = 80
}

resource "azurerm_lb_rule" "web_rule" {
  loadbalancer_id      = azurerm_lb.web_lb.id
  name                 = "HTTPRule"
  protocol              = "Tcp"
  frontend_port        = 80
  backend_port         = 80
  frontend_ip_configuration_name = "PublicIPAddress"
  backend_address_pool_ids = [azurerm_lb_backend_address_pool.web_pool.id]
  probe_id             = azurerm_lb_probe.web_probe.id
}
```

55. Define a VM Scale Set attached to that backend pool:

```

resource "azurerm_linux_virtual_machine_scale_set" "web_vmss" {
  name                = "lab3-vmss"
  resource_group_name = azurerm_resource_group.lab_rg.name
  location            = var.location
  sku                 = "Standard_B1s"
  instances           = 2
  admin_username      = "azureuser"

  admin_ssh_key {
    username   = "azureuser"
    public_key = file("~/ssh/id_rsa.pub")
  }

  source_image_reference {
    publisher = "Canonical"
    offer     = "0001-com-ubuntu-server-jammy"
    sku       = "22_04-lts"
    version   = "latest"
  }

  os_disk {
    storage_account_type = "Standard_LRS"
    caching               = "ReadWrite"
  }

  network_interface {
    name            = "vmss-nic"
    primary         = true
    ip_configuration {
      name                          = "internal"
      primary                      = true
      subnet_id                    = azurerm_subnet.lab_subnet.id
      load_balancer_backend_address_pool_ids =
[azurerm_lb_backend_address_pool.web_pool.id]
    }
  }
}

```

56. Define an Azure SQL server and database:

```

resource "azurerm_mssql_server" "web_sql" {
  name                = "lab3-sqlsrv-<unique>"
  resource_group_name = azurerm_resource_group.lab_rg.name
  location            = var.location
  version             = "12.0"
  administrator_login = "sqladmin"
  administrator_login_password = var.sql_password
}

resource "azurerm_mssql_database" "web_db" {
  name      = "lab3db"
  server_id = azurerm_mssql_server.web_sql.id
  sku_name  = "Basic"
}

```

57. Apply:

```

terraform init
terraform apply -auto-approve

```

✓ Validation Steps

58. `terraform output lb\_public\_ip` returns an IP; `curl <ip>` is load-balanced across scale set instances.

59. Azure Portal → Virtual Machine Scale Sets → confirm 2 healthy instances behind the Load Balancer.

60. Manually deallocate one VMSS instance and confirm the scale set brings the instance count back to the configured value (same self-healing concept validated on AWS in AWS Lab 3).
61. Azure Portal → SQL Databases → confirm `lab3db` is `Online`, and that its firewall blocks public access by default.
62. Confirm the storage account container shows the `lab3.terraform.tfstate` file after apply.
63. `terraform destroy -auto-approve` to remove the entire stack.

LAB 1

Multi-Cloud — Single Config, Two Clouds

Difficulty: Basic (Multi-Cloud)

Objective: Use a single Terraform configuration with two providers to provision one VM in AWS and one VM in Azure at the same time, demonstrating that Terraform is cloud-agnostic at the tooling level.

Steps

64. Create the project:

```
mkdir ~/terraform-mc-lab1 && cd ~/terraform-mc-lab1
touch main.tf
```

65. Declare both providers in the same file:

```
terraform {
  required_providers {
    aws = {
      source = "hashicorp/aws"
      version = "~> 5.0"
    }
    azurerm = {
      source = "hashicorp/azurerm"
      version = "~> 3.0"
    }
  }
}

provider "aws" {
  region = "ap-south-1"
}

provider "azurerm" {
  features {}
}
```

66. Add one resource block per cloud:

```
resource "aws_instance" "aws_node" {
  ami           = "ami-0f5ee92e2d63afc18"
  instance_type = "t2.micro"
  tags         = { Name = "MC-Lab1-AWS" }
}

resource "azurerm_resource_group" "az_rg" {
  name     = "mc-lab1-rg"
  location = "Central India"
}

# ...azurerm_virtual_network / subnet / nic / vm blocks, same pattern as Azure Lab 1...
```

67. Apply both in a single command:

```
terraform init
terraform plan
terraform apply -auto-approve
```

✓ Validation Steps

68. AWS Console shows an EC2 instance tagged `MC-Lab1-AWS` running.

69. Azure Portal shows the `mc-lab1-rg` resource group with a running VM.

70. `terraform state list` shows resources from both the `aws` and `azurerm` providers in one state file.

71. `terraform destroy -auto-approve` removes resources in both clouds from a single command.

LAB 2

Multi-Cloud — Active-Passive DR Across AWS & Azure

Difficulty: Advanced (Multi-Cloud)

Objective: Design a simple active-passive disaster recovery pattern — a primary web app on AWS and a standby copy on Azure — using Terraform modules and DNS-based failover, tying together everything from the earlier labs.

Steps

72. Organize the project into modules:

```
terraform-mc-lab2/
├── main.tf
├── modules/
│   ├── aws_app/      # reuses AWS Lab 3 pattern (ALB + ASG)
│   └── azure_app/     # reuses Azure Lab 3 pattern (LB + VMSS)
```

73. Call each module from the root main.tf:

```
module "primary_aws" {
  source      = "../modules/aws_app"
  region      = "ap-south-1"
  instance_count = 2
}

module "standby_azure" {
  source      = "../modules/azure_app"
  location    = "Central India"
  instance_count = 1 # smaller standby footprint
}
```

74. Add a Route 53 record with a failover routing policy pointing primarily at the AWS ALB, with the Azure Load Balancer IP as secondary:

```
resource "aws_route53_health_check" "primary_hc" {
  fqdn      = module.primary_aws.alb_dns_name
  port      = 80
  type      = "HTTP"
  resource_path = "/"
  request_interval = 30
  failure_threshold = 3
}

resource "aws_route53_record" "primary" {
  zone_id = var.hosted_zone_id
  name     = "app.example.com"
  type     = "A"
  set_identifier = "primary"
  failover_routing_policy { type = "PRIMARY" }
  health_check_id = aws_route53_health_check.primary_hc.id
  alias {
    name            = module.primary_aws.alb_dns_name
    zone_id         = module.primary_aws.alb_zone_id
    evaluate_target_health = true
  }
}

resource "aws_route53_record" "secondary" {
  zone_id = var.hosted_zone_id
  name     = "app.example.com"
  type     = "A"
  ttl      = 60
  set_identifier = "secondary"
  failover_routing_policy { type = "SECONDARY" }
```

```
    records = [module.standby_azure.lb_public_ip]
}
```

75. Apply the full stack:

```
terraform init
terraform plan
terraform apply -auto-approve
```

✓ Validation Steps

76. `dig app.example.com` resolves to the AWS ALB while the primary health check is healthy.
77. `curl http://app.example.com` returns the response from the AWS-hosted app.
78. Simulate a primary failure: stop the AWS ASG instances (set `desired_capacity = 0`) and wait for the Route 53 health check to mark the primary unhealthy.
79. Re-run `dig app.example.com` and confirm DNS now resolves to the Azure Load Balancer's public IP (secondary).
80. `curl http://app.example.com` now returns the response from the Azure-hosted standby, confirming failover works end to end.
81. Restore the AWS ASG (`desired_capacity` back to 2) and confirm Route 53 fails back to primary once the health check passes again.
82. `terraform destroy -auto-approve` to tear down both the AWS and Azure stacks together.

Final Validation Checklist

Use this consolidated checklist to confirm every lab was completed correctly, and that no resources were left running (avoiding unexpected cloud bills).

General Terraform Hygiene (every lab)

- ``terraform fmt`` was run and the code is consistently formatted.
- ``terraform validate`` returns “Success! The configuration is valid.”
- ``terraform plan`` was reviewed before every ``apply`` — no unexpected resource changes.
- ``terraform state list`` matches the resources you expect to exist.
- ``terraform destroy`` was run at the end of every lab.

AWS-Specific Checks

- AWS Console → EC2, VPC, Auto Scaling Groups, and RDS all show zero leftover resources after destroy.
- S3 state bucket and DynamoDB lock table are the only persistent resources left (by design) after Lab 3.
- ``aws ec2 describe-instances`` returns no instances in a ``running`` state that you didn't intend to keep.

Azure-Specific Checks

- Azure Portal → Resource Groups shows only the ``tfstate-rg`` group left (by design) after Azure Lab 3.
- ``az vm list -o table`` and ``az vmss list -o table`` return empty for the lab resource groups after destroy.
- Azure SQL server/database no longer appears in the portal after destroy.

Multi-Cloud Checks

- Both AWS and Azure consoles were checked independently — a single ``terraform destroy`` was confirmed to clean up both clouds.
- DNS failover (Multi-Cloud Lab 2) was tested in both directions: failover to standby, and fail-back to primary.
- No Route 53 hosted zone charges or dangling health checks remain if the domain was only for lab purposes.

Cost Reminder: Always run `terraform destroy` at the end of a session, even between class breaks. Auto Scaling Groups, VM Scale Sets, RDS, and Azure SQL are the most common sources of surprise charges if left running.

End of Lab Guide